Zagazig University

Faculty of Engineering

Computer and Systems Engineering Program

Course: CSE 226: Hardware Design

PROJECT REPORT

Implement Cryptographic Coprocessor 16-bit

Under the Supervision of

Dr. Howida Abd AlLatif

Team Members

No.	Name	seat no	Email
1	Shaza Walid Farid Mohamed	3003	shazawaleed2023@gmail.com
2	Abdullah Ashraf Ahmed El-Sayed	3004	abdallhdawi3@gmail.com
3	Osman Mohamed Salem Ibrahim	3005	mohmedeprahim56@gmail.com
4	Ali Mohamed Abu Khaleel pendary	3006	alikhaleelironside666@gmail.com
5	Ali Mohamed Ali Hussein Ali	3007	aly200565@gmail.com
6	Youssef Ahmed Ibrahim Abdel-Aal	3036	YA2005.5.30@gmail.com

Contents

1 The Assignment Aim **2**

2 The Problem Solution **2**
2.1 Inputs and Outputs 2
2.2 Architecture Overview 2

3 Implementation **3**
3.1 VHDL Code Description and Implementation 3
3.1.1 1. Input Register Unit 3
3.1.2 2. S-Box Unit (Nonlinear Lookup Operation) 5
3.1.3 3. Shifter Unit 7
3.1.4 4. ALU Unit 8
3.1.5 5. Combinational Logic Block (CLU) 9
3.1.6 6. 16×16 Register File 12
3.1.7 7. Top-Level Entity (Cryptographic Coprocessor) 14
3.2 Schematic Diagram 16
3.3 Simulation Results 17
3.3.1 Test Case 1: Input Register Operation 17
3.3.2 Test Case 2: S-Box Operation 17
3.3.3 Test Case 3: Shift Operations 18
3.3.4 Test Case 4: ALU Operations 18
3.3.5 Test Case 5: Register File Operations 19
3.3.6 Test Case 6: Top-Level Integration (CLU & Datapath) 19

4 Project Task Distribution **20**

5 References **20**

1 The Assignment Aim

The primary aim of this project is to design and implement a 16-bit Cryptographic Coprocessor using VHDL. The coprocessor provides standard arithmetic and logical instructions alongside dedicated functional units specific for security applications. The design follows a strict modular approach, integrating a Register File with a Combinational Logic Unit (CLU) that encapsulates a Nonlinear Lookup Operation Unit (S-Boxes), an ALU, and a Shifter to handle specific cryptographic permutations.

2 The Problem Solution

2.1 Inputs and Outputs

Based on the `crypto_coprocessor` top-level entity, the system defines specific control and data signals to manage operations.

Inputs:

- `clock`: Synchronization signal for the input register and sequential memory logic.
- `reset`: Active-high synchronous reset to clear the register file.
- `CTRL`: 4-bit control signal decoded by the Control Logic to select operations (e.g., ALU function, shift type, or S-Box routing).
- `Ra`, `Rb`: 4-bit addresses for the source registers supplying the operands.
- `Rd`: 4-bit address for the destination register where the computed result is stored.

Internal Signals / Outputs:

- `Result`: An internal 16-bit computed data bus outputted from the Combinational Logic Unit (`CLU`) and routed back to the Register File's write input.

2.2 Architecture Overview

The architecture consists of two main interacting sections:

1. **Memory & Input Layer:** Includes a synchronous Input Register for pipelining control signals and generating write-enables, alongside a 16×16 `register_file` that provides dual-read outputs (`SRCa` and `SRCb`) mapped to the data buses.
2. **Combinational Logic Unit (CLU):** A master wrapper module containing the core processing units:
    - **Nonlinear Lookup Operation Unit:** Takes a full 16-bit input, passes the upper byte through unchanged, and performs cryptographic substitution on the lower byte's nibbles via S-Box 1 and S-Box 2.
    - **ALU:** Handles 8 standard operations including ADD, SUB, bitwise logic (AND, OR, XOR, NOT), and pass-through (MOV).

- **Shifter:** Performs precise bitwise manipulations (8-bit right rotate, 4-bit right rotate, 8-bit left shift).
- **Control Logic & MUX:** The Control Logic decodes the 4-bit `CTRL` into specific sub-unit signals, while a Multiplexer routes the active unit's output to the final `RESULT`.

A 16-bit datapath was selected to match project requirements while maintaining manageable hardware complexity.

Modular separation between ALU, Shifter, Control Logic, and S-Box simplifies testing and integration inside the overarching `CLU` wrapper.

3 Implementation

3.1 VHDL Code Description and Implementation

To ensure clarity and maintainability, the VHDL design was constructed modularly. Each component was designed as a separate entity and then instantiated within higher-level wrappers. Below is the description and VHDL implementation for each module.

3.1.1 1. Input Register Unit

The input register module serves as a synchronous pipeline latch at the front end of the coprocessor. It isolates the Combinational Logic Unit (CLU) and the write-back phase from asynchronous input changes by latching the control instruction (`CTRL_IN`) and the destination address (`Rd_IN`) on the rising edge of the clock.

Additionally, this module contains the decoding logic necessary to generate the Register File Write Enable signal (`RdWEn`). It asserts `RdWEn` high for all active operations but pulls it low when a NOP instruction ("0111") is detected, ensuring that memory is not inadvertently overwritten during idle cycles. A synchronous active-high reset is included to force the system into a safe NOP state and clear the destination address.

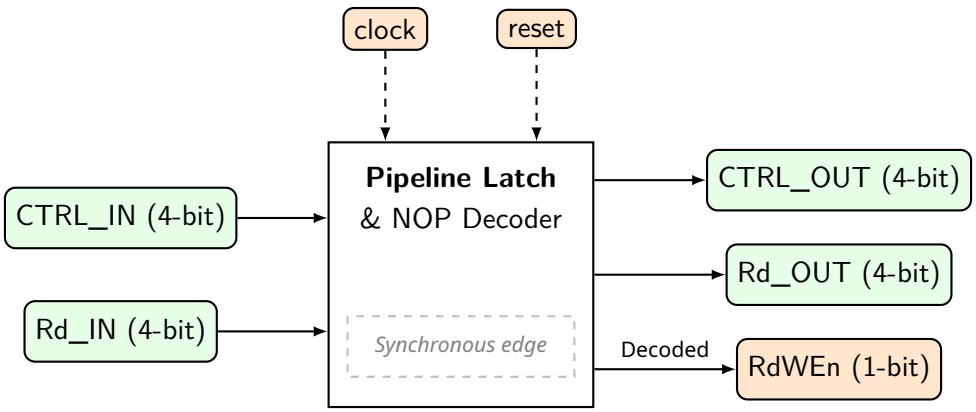

Figure 1: Block diagram of the Input Register and Decoding logic.

Listing 1: VHDL Code for Input Register Unit

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity input_register is
    port (
        clock    : in  std_logic;
        reset    : in  std_logic;
        CTRL_IN  : in  std_logic_vector(3 downto 0);
        Rd_IN    : in  std_logic_vector(3 downto 0);
        CTRL_OUT : out std_logic_vector(3 downto 0);
        Rd_OUT   : out std_logic_vector(3 downto 0);
        RdWEn    : out std_logic
    );
end entity;

architecture rtl of input_register is
    signal RdWEn_reg : std_logic := '0';
begin
    process(clock)
    begin
        if rising_edge(clock) then
            if reset = '1' then
                CTRL_OUT  <= "0111"; -- NOP
                Rd_OUT    <= "0000";
                RdWEn_reg <= '0';
            else
                CTRL_OUT <= CTRL_IN;
                Rd_OUT   <= Rd_IN; -- Delay Rd to match CTRL

                -- RdWEn logic remains synchronous to prevent spikes
                if CTRL_IN = "0111" then
                    RdWEn_reg <= '0';
                else
                    RdWEn_reg <= '1';
                end if;
            end if;
        end if;
    end process;
    RdWEn <= RdWEn_reg;
end architecture;
```

3.1.2 2. S-Box Unit (Nonlinear Lookup Operation)

The `non_linear_lookup` unit is responsible for cryptographic substitution. Unlike a simple 8-bit unit, the implemented VHDL module processes a full 16-bit input (`LUTIN`). It splits this input: the upper byte (bits 15 downto 8) is passed through unchanged, while the lower byte (bits 7 downto 0) is divided into two 4-bit nibbles. The Most Significant Nibble (MSN) is substituted using S-Box 1, and the Least Significant Nibble (LSN) is substituted using S-Box 2 via efficient VHDL `case` statements. Finally, the substituted nibbles are combined with the unmodified upper byte to form the 16-bit `LUTOUT`.

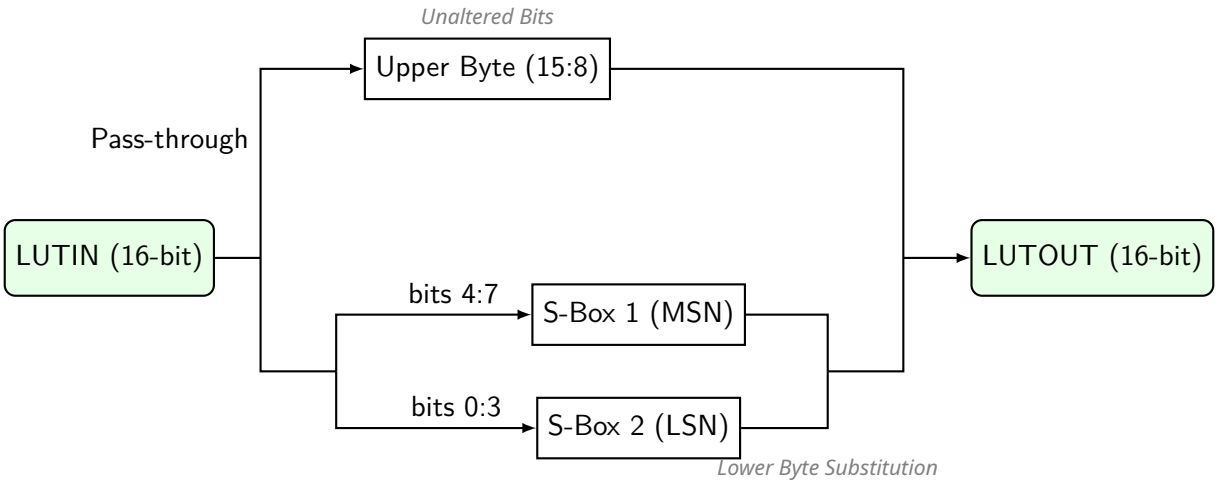

Figure 2: Internal Architecture of the Nonlinear Lookup Operation Unit (S-Box).

Listing 2: VHDL Code for S-Box Unit

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

--16-bit Nonlinear Lookup(Integrated Version)
entity non_linear_lookup is
    port (
        LUTIN  : in  std_logic_vector(15 downto 0); --full 16-bit input
        LUTOUT : out std_logic_vector(15 downto 0)  --full 16-bit output
    );
end entity;

architecture rtl of non_linear_lookup is

    -- Split signals
    signal upper_byte       : std_logic_vector(7 downto 0); -- unchanged part
    signal lower_byte       : std_logic_vector(7 downto 0); -- part to process
    signal MSN_in, LSN_in   : std_logic_vector(3 downto 0);
    signal MSN_out, LSN_out : std_logic_vector(3 downto 0);
    signal lut_out          : std_logic_vector(7 downto 0); --the output of s-boxes

    begin
        -- Step 1: Split input into upper and lower bytes
        upper_byte <= LUTIN(15 downto 8); -- pass-through
        lower_byte <= LUTIN(7 downto 0);  -- will go through LUT
        -- Step 2: Split lower byte into two nibbles
```

```
        MSN_in <= lower_byte(7 downto 4); -- upper nibble
        LSN_in <= lower_byte(3 downto 0); -- lower nibble
        -- Step 3: S-Box1 (for MSN)
        --This is a lookup table implemented using a case statement
        --Case statement is used because S-Box is a lookup table,and case
            directly represents mapping between input and output values(hardware
            efficiency)
        process(MSN_in)
        begin
            case MSN_in is
                when "0000" => MSN_out <= "0001";
                when "0001" => MSN_out <= "1011";
                when "0010" => MSN_out <= "1001";
                when "0011" => MSN_out <= "1100";
                when "0100" => MSN_out <= "1101";
                when "0101" => MSN_out <= "0110";
                when "0110" => MSN_out <= "1111";
                when "0111" => MSN_out <= "0011";
                when "1000" => MSN_out <= "1110";
                when "1001" => MSN_out <= "1000";
                when "1010" => MSN_out <= "0111";
                when "1011" => MSN_out <= "0100";
                when "1100" => MSN_out <= "1010";
                when "1101" => MSN_out <= "0010";
                when "1110" => MSN_out <= "0101";
                when "1111" => MSN_out <= "0000";
                when others => MSN_out <= "0000";  --safity default
            end case;
        end process;
        -- Step 4: S-Box2 (for LSN)
        process(LSN_in)
        begin
            case LSN_in is
                when "0000" => LSN_out <= "1111";
                when "0001" => LSN_out <= "0000";
                when "0010" => LSN_out <= "1101";
                when "0011" => LSN_out <= "0111";
                when "0100" => LSN_out <= "1011";
                when "0101" => LSN_out <= "1110";
                when "0110" => LSN_out <= "0101";
                when "0111" => LSN_out <= "1010";
                when "1000" => LSN_out <= "1001";
                when "1001" => LSN_out <= "0010";
                when "1010" => LSN_out <= "1100";
                when "1011" => LSN_out <= "0001";
                when "1100" => LSN_out <= "0011";
                when "1101" => LSN_out <= "0100";
                when "1110" => LSN_out <= "1000";
                when "1111" => LSN_out <= "0110";
                when others => LSN_out <= "0000"; --safity default
            end case;
        end process;
        -- Step 5: Combine S-Box outputs
        lut_out <= MSN_out & LSN_out;
        -- Step 6: Concatenate with upper byte to form final 16-bit output
        LUTOUT <= upper_byte & lut_out;

end architecture;
```

3.1.3 3. Shifter Unit

The `shifter` unit performs customized bitwise shifts on the `SHIFTINPUT` using the IEEE `numeric_std` library. Based on a 2-bit `SHIFT_Ctrl` signal decoded by the Control Logic, it executes an 8-bit right rotation using `rotate_right` for "00", a 4-bit right rotation for "01", and an 8-bit left shift using `shift_left` for "10". Invalid states ("11") yield a cleared output.

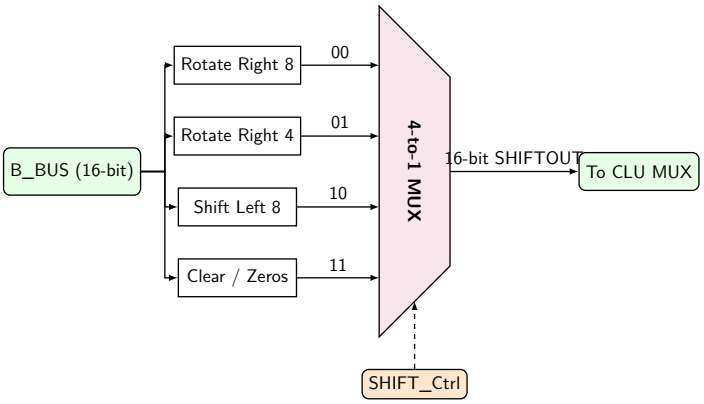

Figure 3: Block diagram of the Shifter Unit.

Listing 3: VHDL Code for Shifter Unit

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use ieee.numeric_std.all;

entity shifter is
    generic (
        N : integer := 16
    );
    port (
        SHIFTINPUT : in  std_logic_vector(N-1 downto 0);
        SHIFT_Ctrl : in  std_logic_vector(1 downto 0);
        SHIFTOUT   : out std_logic_vector(N-1 downto 0)
    );
end entity shifter;
architecture rtl of shifter is
begin
    process(SHIFTINPUT, SHIFT_Ctrl)
    begin
        case SHIFT_Ctrl is
            when "00" =>
                SHIFTOUT <= std_logic_vector(                          ---
                    Rotate input right by 8 bits
                    rotate_right(unsigned(SHIFTINPUT), 8));
            when "01" =>
                SHIFTOUT <= std_logic_vector(                          ---
                    Rotate input right by 4 bits
                    rotate_right(unsigned(SHIFTINPUT), 4));
            when "10" =>
                SHIFTOUT <= std_logic_vector(                          ---
                    Shift input left by 8 bits
                    shift_left(unsigned(SHIFTINPUT), 8));
            when others =>
                SHIFTOUT <= (others => '0');                           ---
                    invalid SHIFT_Ctrl ----> clear output
        end case;
    end process;

end architecture rtl;
```

3.1.4 4. ALU Unit

The Arithmetic Logic Unit (`ALU`) handles standard 16-bit mathematical and bitwise operations between `ABUS` and `BBUS`. Governed by a 3-bit `ALUctrl` signal, it employs a `with ... select` concurrent assignment to execute ADD ("000"), SUB ("001"), AND ("010"), OR ("011"), XOR ("100"), NOT ("101"), MOV ("110"), or NOP ("111").

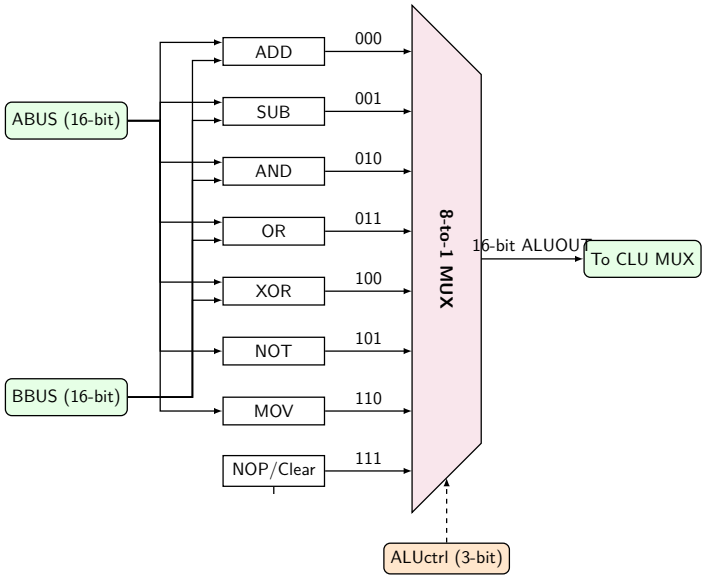

Figure 4: Architecture and Internal Logic of the 16-bit ALU.

Listing 4: VHDL Code for ALU Unit

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

-- ALU module
-- operations: ADD SUB AND OR XOR NOT MOV NOP

entity ALU is
    port (
        ABUS    : in  std_logic_vector(15 downto 0);
        BBUS    : in  std_logic_vector(15 downto 0);
        ALUctrl : in  std_logic_vector(2 downto 0);
        ALUOUT  : out std_logic_vector(15 downto 0)
    );
end entity ALU;

architecture rtl of ALU is
begin

   with ALUctrl select
     ALUOUT <= std_logic_vector(signed(ABUS) + signed(BBUS)) when "000",
               std_logic_vector(signed(ABUS) - signed(BBUS)) when "001",
               ABUS AND BBUS                                 when "010",
               ABUS OR BBUS                                  when "011",
               ABUS XOR BBUS                                 when "100",
               NOT ABUS                                      when "101",
               ABUS                                          when "110",
               (others => '0')                               when others;


end architecture rtl;
```

3.1.5 5. Combinational Logic Block (CLU)

The CLU block acts as the master integration wrapper. It instantiates the Control_Logic to decode the 4-bit CTRL input into dedicated ALUctrl, SHIFTctrl, and MUXctrl signals. The operands are routed simultaneously to the processing units: both A_BUS and B_BUS feed into the ALU, while the shifter processes only B_BUS, and the non_linear_lookup processes only A_BUS. Finally, a dedicated MUX module multiplexes their outputs, selecting the active unit's result to drive the RESULT port.

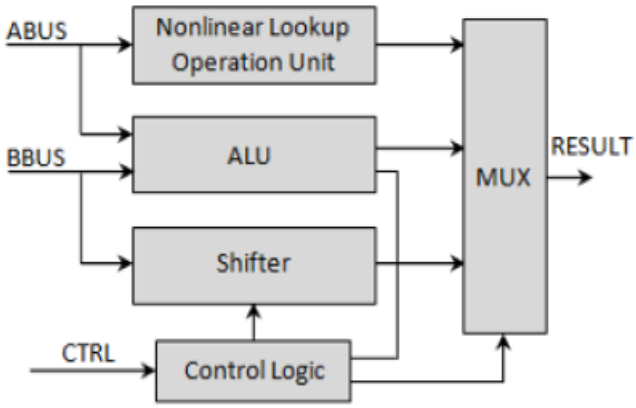

Figure 5: Architecture of the Combinational Logic block.

Listing 5: VHDL Code for Control Logic

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Control_Logic is
    port (
        CTRL          : in  std_logic_vector(3 downto 0); -- From Input Register
        ALU_Ctrl      : out std_logic_vector(2 downto 0); -- To ALU (3 bits)
        SHIFT_Ctrl    : out std_logic_vector(1 downto 0); -- To Shifter (2 bits)
        MUX_SEL       : out std_logic_vector(1 downto 0)  -- To MUX (2 bits)
    );
end entity Control_Logic;

architecture rtl of Control_Logic is
begin
    -- This process decodes the "What" and "Where"
    process(CTRL)
    begin
        -- Default assignments to avoid latches
        ALU_Ctrl    <= "000";
        SHIFT_Ctrl <= "00";
        MUX_SEL     <= "00";

        if CTRL(3) = '0' then
            -- ALU Operations (0000 to 0111)
            -- As per TA instructions: ALU gets only the lower 3 bits
            ALU_Ctrl <= CTRL(2 downto 0);
            MUX_SEL  <= "01"; -- Route ALU to Result

        else
            -- Dedicated Security/Shift Operations (1000 to 1011)
            case CTRL is
                when "1000" => -- ROR8
                    SHIFT_Ctrl <= "00";
```

```
                    MUX_SEL     <= "10";
                when "1001" => -- ROR4
                    SHIFT_Ctrl <= "01";
                    MUX_SEL     <= "10";
                when "1010" => -- SLL8
                    SHIFT_Ctrl <= "10";
                    MUX_SEL     <= "10";
                when "1011" => -- Nonlinear LUT
                    MUX_SEL     <= "11";
                when others =>
                    -- NOP / Invalid Case
                    MUX_SEL     <= "00"; -- Result will be 0x0000
            end case;
        end if;
    end process;
end architecture;
```

Listing 6: VHDL Code for Multiplexer (MUX)

```
library ieee;
use ieee.std_logic_1164.all;

entity MUX is
    port(
        ALU_OUT,SHIFT_OUT,LUT_OUT : in std_logic_vector(15 downto 0);
        SEL                       : in std_logic_vector(1 downto 0);
        MUX_OUT                   : out std_logic_vector(15 downto 0)
    );
end entity;

architecture rtl of MUX is
begin

    MUX_OUT <=  ALU_OUT when SEL = "01" else
                SHIFT_OUT when SEL = "10" else
                LUT_OUT when SEL = "11" else
                x"0000";

end architecture;
```

Listing 7: VHDL Code for Combinational Logic Block (CLU)

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Combinational Logic Unit (CLU)
-- Integrates Control_Logic, ALU, Shifter, LUT, and MUX

entity CLU is
    port (
    A_BUS  : in  std_logic_vector(15 downto 0); -- First operand
    B_BUS  : in  std_logic_vector(15 downto 0); -- Second operand
    CTRL   : in  std_logic_vector(3 downto 0);  -- Control signal (operation
        selector)
    RESULT : out std_logic_vector(15 downto 0)  -- Final output
    );
end entity;

```

```vhdl
architecture rtl of CLU is

    -- Internal Control Signals
    signal ALUctrl     : std_logic_vector(2 downto 0);
    signal SHIFTctrl   : std_logic_vector(1 downto 0);
    signal MUXctrl     : std_logic_vector(1 downto 0);

    -- Internal Data Signals (Outputs from sub-units)
    signal ALUout     : std_logic_vector(15 downto 0);
    signal SHIFTout   : std_logic_vector(15 downto 0);
    signal LUTout     : std_logic_vector(15 downto 0);

begin
    -- 1. Control Logic
    -- Takes 4-bit CTRL and tells the other units what to do
    CTRL_inst: entity work.Control_Logic
        port map (
            CTRL       => CTRL,
            ALU_Ctrl   => ALUctrl,
            SHIFT_Ctrl => SHIFTctrl,
            MUX_SEL    => MUXctrl
        );

    -- 2. ALU
    ALU_inst: entity work.ALU
        port map (
            ABUS    => A_BUS,
            BBUS    => B_BUS,
            ALUctrl => ALUctrl,
            ALUOUT  => ALUout
        );

    -- 3. Shifter
    SHIFTER_inst: entity work.shifter
        port map (
            SHIFTINPUT => B_BUS,
            SHIFT_Ctrl => SHIFTctrl,
            SHIFTOUT   => SHIFTout
        );

    -- 4. Nonlinear Lookup (LUT)
    LUT_inst: entity work.non_linear_lookup
        port map (
            LUTIN  => A_BUS,
            LUTOUT => LUTout
        );

    -- 5. Multiplexer (MUX)
    MUX_inst: entity work.MUX
        port map (
            ALU_OUT   => ALUout,
            SHIFT_OUT => SHIFTout,
            LUT_OUT   => LUTout,
            SEL       => MUXctrl,
            MUX_OUT   => RESULT
        );

end architecture;
```

3.1.6 6. 16×16 Register File

The `register_file` memory unit consists of a custom VHDL `reg_array` type containing sixteen 16-bit registers. It employs combinational read logic for zero-latency outputs (`SRCa`, `SRCb`) driven by read addresses `Ra` and `Rb`. Conversely, the write operation is sequential; on the rising edge of the `clock`, data (`RES`) is written to the destination address (`Rd`) strictly if the Write Enable (`RdWEn`) is asserted. It also features a synchronous active-high `reset` that restores all memory blocks to a predefined `INITIAL_STATE`. Rather than clearing the registers to `0x0000`, this reset state pre-loads the memory array with specific non-zero constants (e.g., `0xC505` in R1, `0x3C07` in R2) required for cryptographic operations and simulation verification.

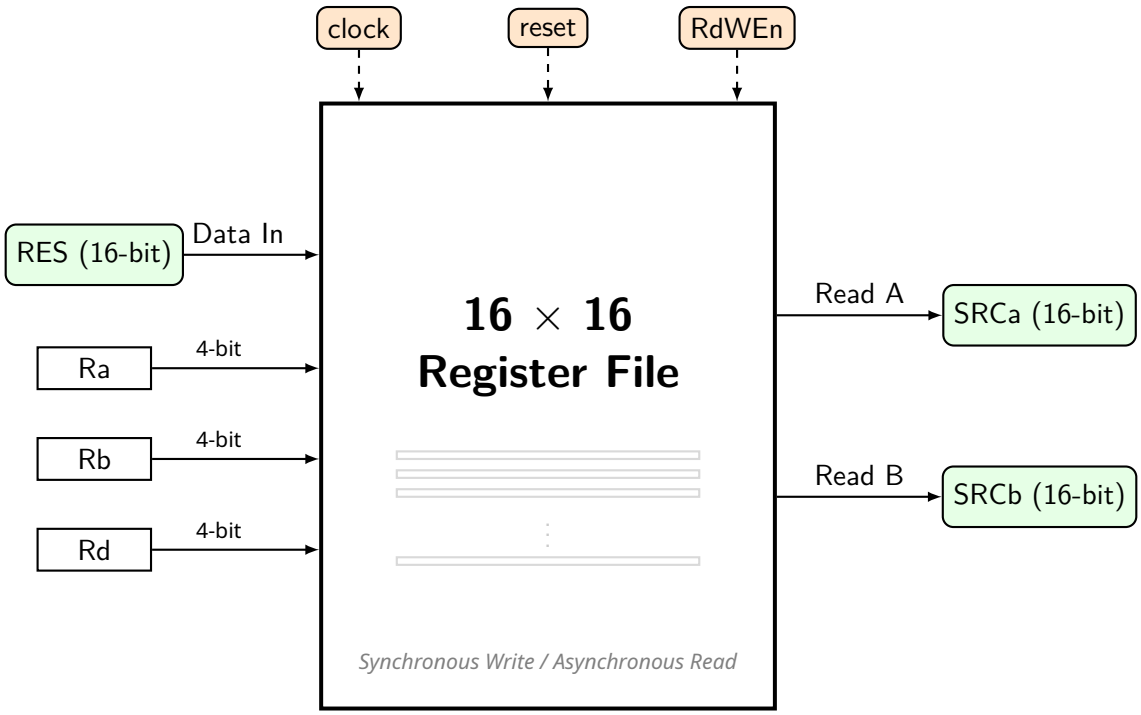

Figure 6: Internal Architecture of the 16×16 Register File Unit.

Listing 8: VHDL Code for Register File

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

-- 16x16 Register File: Defines a storage block for 16 registers of 16-bit width
entity register_file is
    Port (
        clock : in  STD_LOGIC;
        reset : in  STD_LOGIC;
        RdWEn : in  STD_LOGIC;
        RES   : in  STD_LOGIC_VECTOR (15 downto 0);
        Ra    : in  STD_LOGIC_VECTOR (3 downto 0);
        Rb    : in  STD_LOGIC_VECTOR (3 downto 0);
        Rd    : in  STD_LOGIC_VECTOR (3 downto 0);
        SRCa  : out STD_LOGIC_VECTOR (15 downto 0);
        SRCb  : out STD_LOGIC_VECTOR (15 downto 0)
    );
end entity;

architecture rtl of register_file is

    -- Internal storage structure definition
    type reg_array is array (0 to 15) of STD_LOGIC_VECTOR(15 downto 0);

    -- Reset values and initial power-up state
    constant INITIAL_STATE : reg_array := (
        0 => x"0001", 1 => x"C505", 2 => x"3C07", 3 => x"4D05",
        4 => x"1186", 5 => x"F407", 6 => x"1086", 7 => x"4706",
        8 => x"6808", 9 => x"BAA0", 10 => x"C902", 11 => x"100B",
        12 => x"C000", 13 => x"C902", 14 => x"100B", 15 => x"B000"
    );

    signal registers : reg_array := INITIAL_STATE;

begin
    -- Sequential Write Logic: Handles reset and write-enable on clock edges
    process(clock)
    begin
        if rising_edge(clock) then
            if reset = '1' then
                registers <= INITIAL_STATE;
            elsif RdWEn = '1' then
                registers(to_integer(unsigned(Rd))) <= RES;
            end if;
        end if;
    end process;

    -- Combinational Read Logic: Provides concurrent access to two registers
    SRCa <= registers(to_integer(unsigned(Ra)));
    SRCb <= registers(to_integer(unsigned(Rb)));

end architecture;
```

3.1.7 7. Top-Level Entity (Cryptographic Coprocessor)

The `crypto_coprocessor` top-level module forms the macro-architecture by declaring the `register_file` and `CLU` components. It establishes the external interface with `clock`, `reset`, `CTRL`, `Ra`, `Rb`, and `Rd` ports. Internal signals are declared to complete the system loop, bridging the `CLU`'s `RESULT` back to the `RES` input of the Register File, awaiting final structural port mapping and input register latching logic.

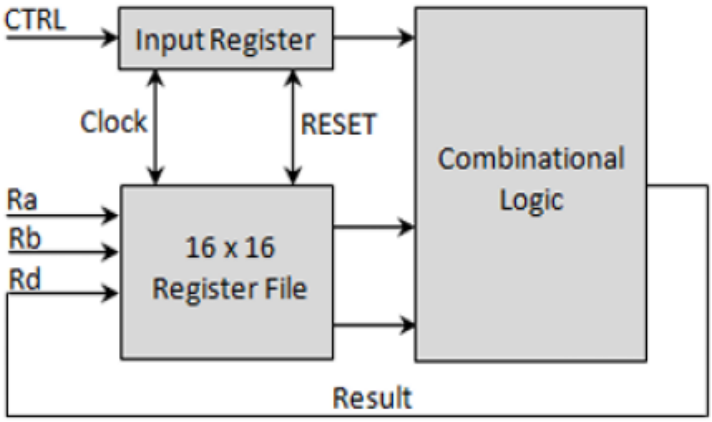

Figure 7: Overall Architecture of the Cryptographic Coprocessor.

Listing 9: VHDL Code for the Top-Level Entity

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Cryptographic Coprocessor (Top Level) -- matches Figure 1 architecture
-- Ra, Rb, Rd feed the register file directly (concurrent read, no clock needed)
    .
-- Only CTRL passes through the input_register pipeline latch.
--
-- Pipeline (one cycle latency on CTRL only):
--   Cycle N  rising edge : input_register latches CTRL_IN ? CTRL_reg
--   After  N (combinat.) : register_file reads Ra/Rb directly ? ABUS/BBUS
--                          CLU uses CTRL_reg + ABUS/BBUS ? RESULT
--   Cycle N+1 rising edge: register_file writes RESULT to Rd (if RdWEn='1')
--
-- NOTE: Rd must be held stable across both edges for correct write-back.
--       In back-to-back instructions the stimulus should not change Rd_IN
--       until after the write-back edge.

entity crypto_coprocessor is
    port (
        clock    : in  std_logic;
        reset    : in  std_logic;
        CTRL_IN  : in  std_logic_vector(3 downto 0);
        Ra_IN    : in  std_logic_vector(3 downto 0);
        Rb_IN    : in  std_logic_vector(3 downto 0);
        Rd_IN    : in  std_logic_vector(3 downto 0);
        -- Observation ports (for testbench / debug)
        A_OUT    : out std_logic_vector(15 downto 0);  -- SRCa (value at Ra)
        B_OUT    : out std_logic_vector(15 downto 0);  -- SRCb (value at Rb)
```

```vhdl
        RESULT  : out std_logic_vector(15 downto 0)   -- CLU output / write-back data
    );
end entity crypto_coprocessor;

architecture rtl of crypto_coprocessor is

    -- Registered control signal (from input_register)
    signal CTRL_reg   : std_logic_vector(3 downto 0);
    signal RdWEn      : std_logic;
    signal Rd_reg : std_logic_vector(3 downto 0);

    -- Data buses between register_file and CLU
    signal SRCa       : std_logic_vector(15 downto 0);
    signal SRCb       : std_logic_vector(15 downto 0);
    signal CLU_RESULT : std_logic_vector(15 downto 0);

begin
    -- 1. Input Register: pipeline latch for CTRL + NOP/reset gating
    --    Ra, Rb, Rd bypass this and go directly to the register file
    IR_inst : entity work.input_register
        port map (
            clock    => clock,
            reset    => reset,
            CTRL_IN  => CTRL_IN,
            Rd_IN    => Rd_IN,
            CTRL_OUT => CTRL_reg,
            Rd_OUT   => Rd_reg,
            RdWEn    => RdWEn
        );
    -- 2. Register File: 16x16, synchronous write, asynchronous read
    --    Ra/Rb/Rd are wired directly from top-level ports (diagram Figure 1)
    RF_inst : entity work.register_file
        port map (
            clock => clock,
            reset => reset,
            RdWEn => RdWEn,
            RES   => CLU_RESULT,
            Ra    => Ra_IN,
            Rb    => Rb_IN,
            Rd    => Rd_reg,
            SRCa  => SRCa,
            SRCb  => SRCb
        );
    -- 3. Combinational Logic Unit: ALU + Shifter + LUT + MUX
    CLU_inst : entity work.CLU
        port map (
            A_BUS  => SRCa,
            B_BUS  => SRCb,
            CTRL   => CTRL_reg,
            RESULT => CLU_RESULT
        );
    -- Drive observation outputs
    A_OUT  <= SRCa;
    B_OUT  <= SRCb;
    RESULT <= CLU_RESULT;

end architecture rtl;
```

3.2 Schematic Diagram

The following figure demonstrates the system-level schematic generated using Active HDL.

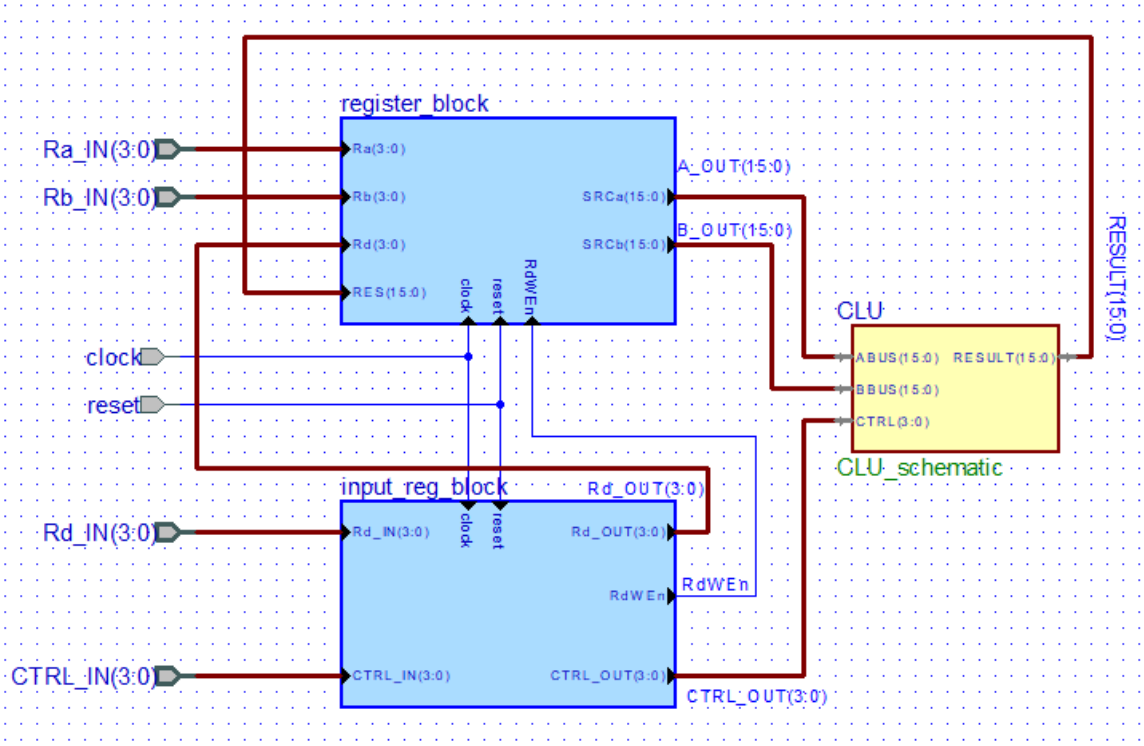

Figure 8: Schematic diagram of the 16-bit Cryptographic Coprocessor.

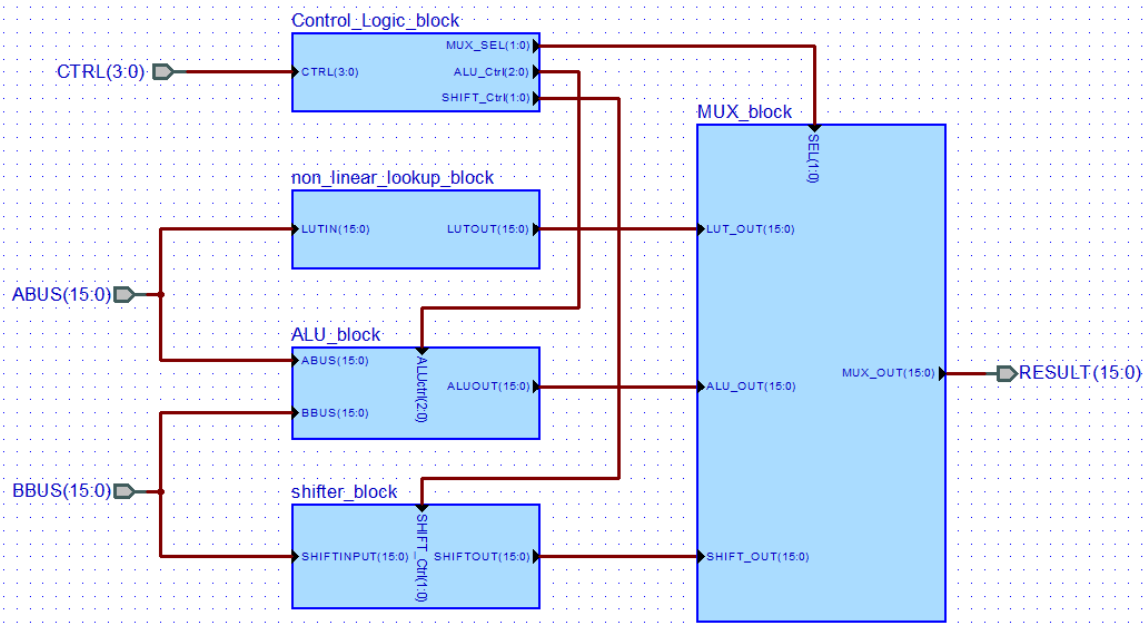

Figure 9: Schematic diagram of the CLU.

3.3 Simulation Results

The design was simulated to verify the correctness of the individual units and the overall coprocessor.

3.3.1 Test Case 1: Input Register Operation

Objective: Validate the synchronous pipelining of control signals and destination addresses, ensure proper generation of the Write Enable (`RdWEn`) signal, and confirm reset behavior.

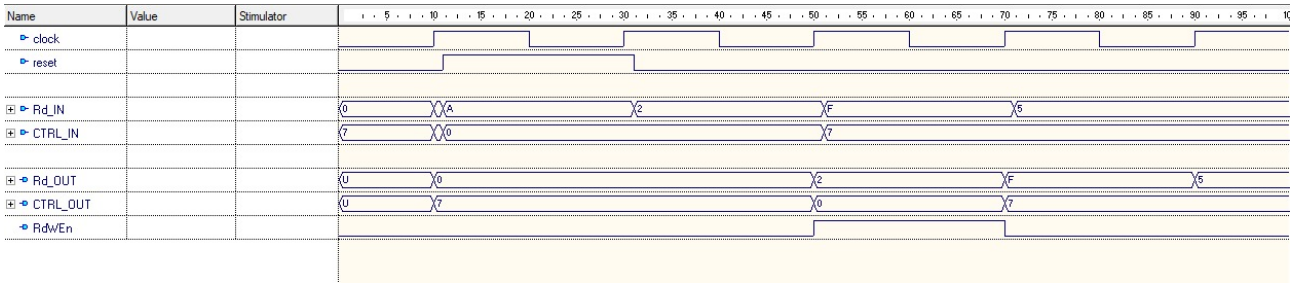

Figure 10: Simulation verifying the Input Register Unit.

Results Analysis:
The testbench successfully verifies the pipeline delay, confirming that `Rd_OUT` and `CTRL_OUT` update strictly on the rising clock edge. When a normal instruction like ADD ("0000") is passed, the module successfully latches the instruction and asserts `RdWEn` to '1'. When a NOP ("0111") is issued, `RdWEn` is correctly driven low to prevent unintended memory overwrites, even while `Rd` continues to latch. The global reset was also verified; it correctly overrides active inputs and forces a safe NOP state (`CTRL_OUT` = "0111", `RdWEn` = '0').

3.3.2 Test Case 2: S-Box Operation

Objective: Validate the Nonlinear Lookup Operation by passing distinct 16-bit test vectors (e.g., `0xABCD`, `0x1234`, `0xFFFF`, `0x0000`) into the `ABUS` and confirming that the lower byte is correctly substituted via S-Box 1 and S-Box 2 while the upper byte remains unaltered.

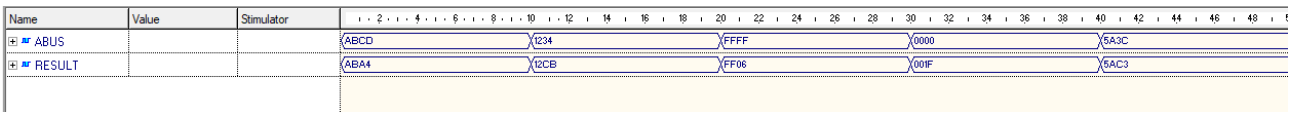

Figure 11: Simulation verifying the Nonlinear Lookup Operation Unit.

Results Analysis:
The simulation rigorously validates the S-Box unit against multiple test vectors. As verified in the testbench, feeding an input of `0xABCD` results in the correct substitution `0xABA4` (upper byte passes through, while `0xCD` transforms via the lookup table). Further test cases corroborate accuracy: `0x1234` yields `0x12CB`, `0xFFFF` correctly resolves to `0xFF06`, and zero input (`0x0000`) successfully translates to `0x001F`. This confirms flawless parallel 4-bit substitution without cross-contamination.

3.3.3 Test Case 3: Shift Operations

Objective: Verify the Shifter module correctly executes 8-bit right rotation (`ROR8`), 4-bit right rotation (`ROR4`), and 8-bit left shift (`SLL8`) on a static input (`0x1234`) across all valid and invalid `SHIFT_Ctrl` permutations.

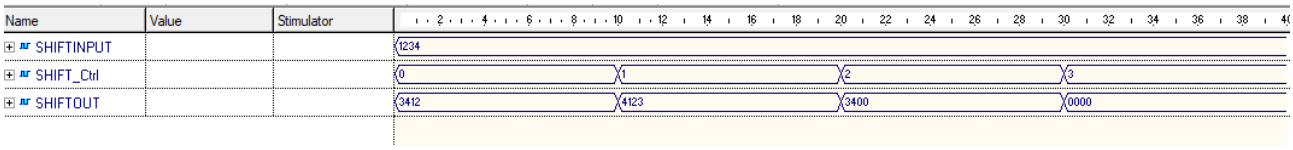

Figure 12: 8-bit Right Rotation Simulation.

Results Analysis:
Using an initial static input of `0x1234`, the shifter module flawlessly demonstrated the required cryptographic permutations. The ROR8 operation accurately wrapped the lower byte to the upper position yielding `0x3412`. The ROR4 operation executed correctly, producing `0x4123`. Finally, the SLL8 logic successfully shifted the upper byte left and padded the rightmost bits with zeros, resulting in `0x3400`. Invalid control states automatically default the output to `0x0000`, ensuring predictable fallbacks.

3.3.4 Test Case 4: ALU Operations

Objective: Test the complete instruction set of the Arithmetic Logic Unit (ADD, SUB, AND, OR, XOR, NOT, MOV, and NOP) by applying discrete 16-bit operand pairs to `ABUS` and `BBUS` and cycling through all eight 3-bit `ALUctrl` codes.

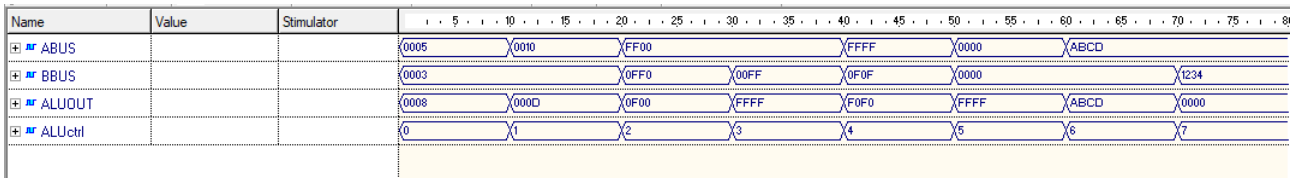

Figure 13: Simulation verifying ALU addition and logical operations.

Results Analysis:
The ALU testbench verifies a comprehensive set of operations. Arithmetic functionality is confirmed with the ADD instruction (`0x0005` + `0x0003` = `0x0008`) and the SUB instruction (`0x0010` - `0x0003` = `0x000D`). Bitwise logic is equally robust; for example, the AND operation on `0xFF00` and `0x0FF0` accurately masks to `0x0F00`, while the XOR logic evaluated `0xFFFF` XOR `0x0F0F` identically to `0xF0F0`. The pass-through (MOV) logic behaves exactly as intended, confirming structural stability.

3.3.5 Test Case 5: Register File Operations

Objective: Confirm the Register File's synchronous write functionality by writing test vectors to registers R1 and R4, test its asynchronous dual-read capabilities, verify write-disable logic to prevent overwriting, and validate the global asynchronous reset.

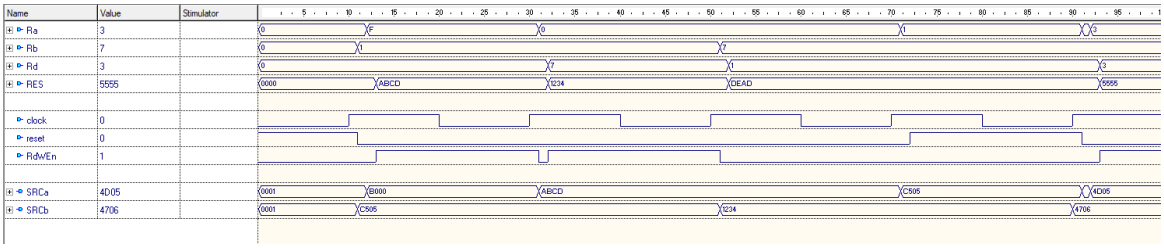

Figure 14: Simulation verifying Data Write and Read from the Register File.

Results Analysis:
Simulations confirm precise clock-synchronized memory allocation. In testing, data `0xC505` was successfully written to Register 1 (`Rd = 1`) and `0x1186` to Register 4 (`Rd = 4`) when the write-enable (`RdWEn`) was active. Attempting to overwrite Register 1 with `0xDEAD` while write-enable was disabled failed exactly as expected, retaining the original `0xC505`. Asynchronous reading via `Ra` and `Rb` was instant and error-free. Finally, triggering the global reset reliably restored all memory registers to their pre-loaded `INITIAL_STATE` test vectors (rather than clearing to zero), behaving exactly as defined in the memory array architecture.

3.3.6 Test Case 6: Top-Level Integration (CLU & Datapath)

Objective: Validate the multiplexed Combinational Logic Unit (CLU) by sequencing through varied `CTRL` signals (`0000`, `0001`, `1000`, `1011`, `1111`) to ensure proper routing and execution among the ALU, Shifter, and LUT components without data collision.

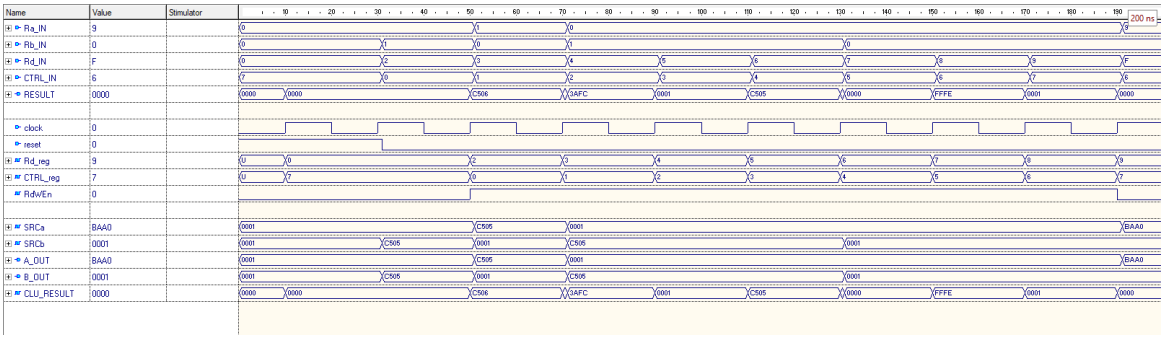

Figure 15: Top-Level integration demonstrating a full execution cycle.

Results Analysis:
The comprehensive CLU testbench validates the cohesive execution of the multiplexed datapath. With test inputs `A_BUS = 0x0005` and `B_BUS = 0x0003`, the ALU ADD control (`0000`) correctly pushed `0x0008` to the result, and SUB (`0001`) pushed `0x0002`. Modifying the B_BUS to `0x1234` and asserting the Shift ROR8 control (`1000`) successfully bypassed the ALU to yield `0x3412` at the final output. The system easily transitioned back to the LUT path with `CTRL = 1011`, delivering the pre-verified S-Box output.

4 Project Task Distribution

To ensure the successful completion of the 16-bit Cryptographic Coprocessor, the project was divided into specific technical and administrative tasks. The following table outlines the responsibilities assigned to each team member during the final development phase:

No.	Name	Task Description
1	**Abdullah Ashraf Ahmed El-Sayed**	• Development of Testbenches (TBs) • Top-Level Coprocessor integration • Input Register implementation
2	**Shaza Walid Farid Mohamed**	• Design of the Combinational Logic Unit (CLU) wrapper • Register File memory logic
3	**Osman Mohamed Salem Ibrahim**	• Implementation of the Control Unit logic • Arithmetic Logic Unit (ALU) design
4	**Ali Mohamed Abu Khaleel Pendary**	• Preparing the report • Nonlinear Lookup (LUT) S-Box unit
5	**Ali Mohamed Ali Hussein Ali**	• Schematic generation in Active-HDL • Multiplexer (MUX) logic design
6	**Youssef Ahmed Ibrahim Abdel-Aal**	• Preparing the Presentation • Shifter Unit logic design

5 References

1. Course CSE 226: Hardware Design Lecture Notes, Zagazig University.
2. Active-HDL documentation